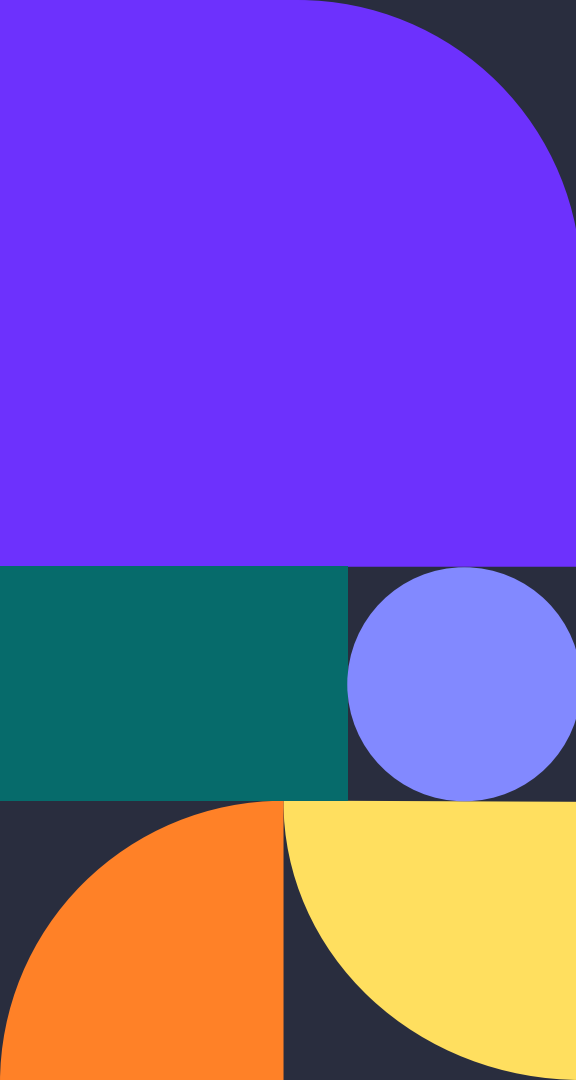




AWQ

Activation-aware Weight Quantization for On-device LLM Compression and Acceleration

Authors: Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, Song Han

Presented by: Buvana Seshathri

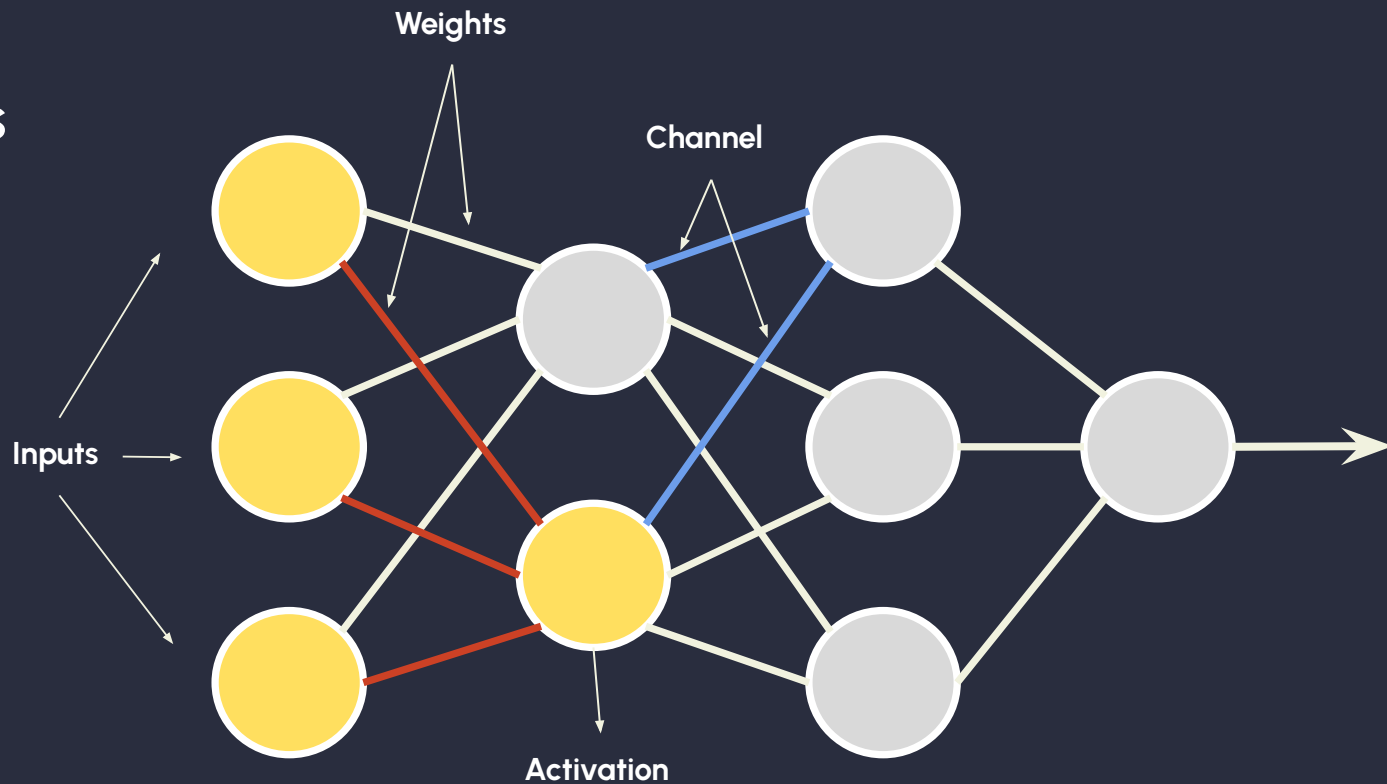
Problem

- Mobile AI Efficiency based Paper
- Deploying LLMs on edge devices
- Need for On-device LLMs?
 - Protect user privacy
 - Reduce Cloud computing costs

Basic Terms

- Weights
- Activation
- Channels
- Quantization
- Dequantization

Weights, Channels, Activations



Model size

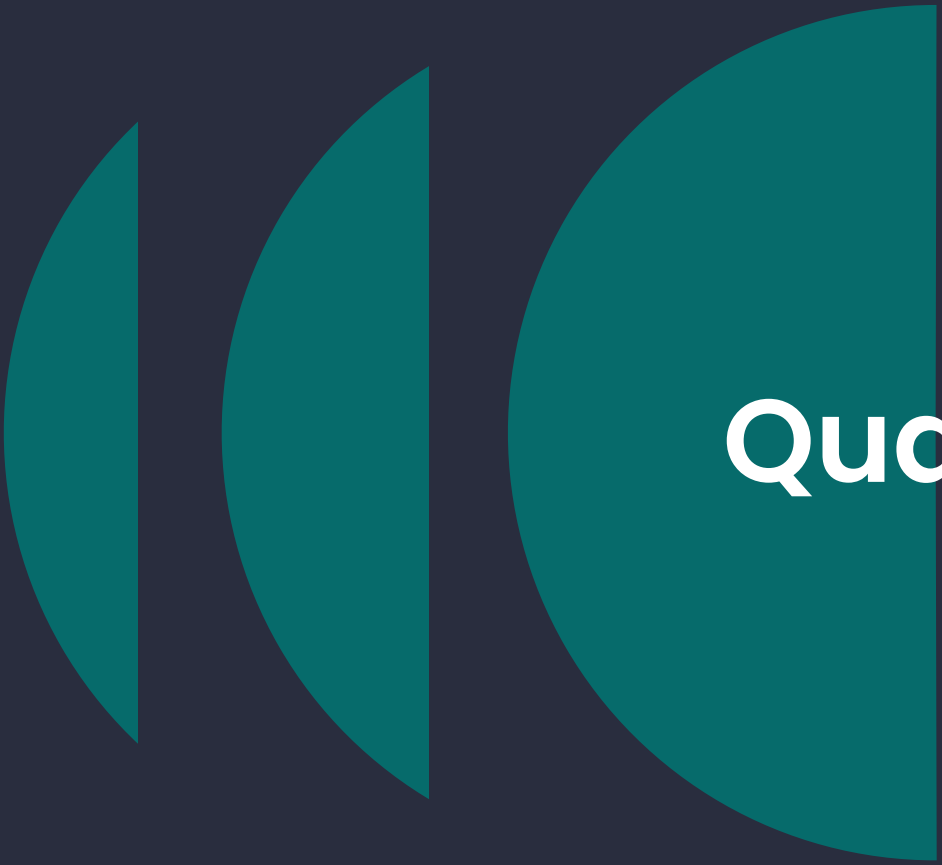
Example weights used by neural networks

- 9.9999999999
- 7.2222222222

What is the problem here?

- These weights are very accurate
- But memory-heavy
- FP32 or FP64 consumes lot of memory
- How many of these weights do we need?

BILLIONS

Three teal semi-circles of increasing size are arranged horizontally on the left side of the slide. The word "Quantization" is written in white, bold, sans-serif font, centered over the largest semi-circle.

Quantization

Quantization

- Instead of 9.99999, just use 10?
- 9.99999 $\rightarrow$ FP32, 4 bytes
- 10 $\rightarrow$ Int, 1 byte (actually just 4 bits)
- What?
Compress weights to fewer bits integers
- Why?
Reduce memory & computation
Models become smaller and faster

Quantization & Dequantization

Consider this example:

4-bit quantization

Let's say range of weights is -1.0 to 1.0

We are trying to quantize **w1 = 0.89083**

Closest 4-bit mapping is 14

So quantized weight is 14

Dequantized weight is 0.8667

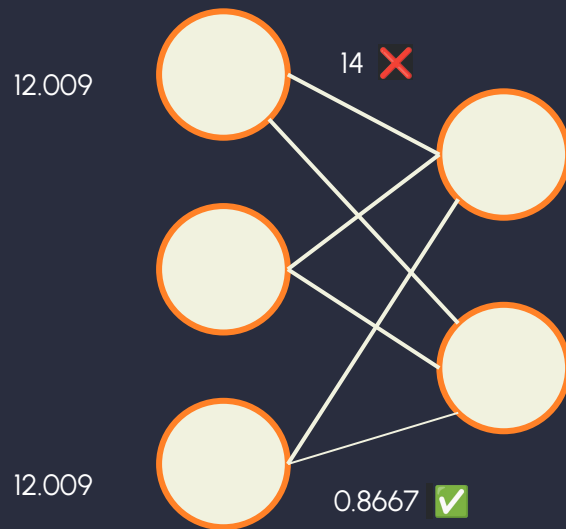
4-bit integer	Dequantized (approx)
0	- 1.0
1	- 0.8667
...	...
14	0.8667
15	1.0

Why do we need Dequantization?

- Most hardware use floating-point forms

- 12.009×14
(input) (quantized weight)

- 12.009×0.8667
(input) (dequantized weight)



Existing Methods

RTN (Round-to-nearest)

- Directly rounds full-precision weights to nearest quantized value
- Very Poor accuracy

PTQ (Post-training Quantization)

- Quantize after training, with an optional dataset
- Poor accuracy

QAT (Quantization-aware Training)

- Simulates quantization during training to adapt weights
- High training costs

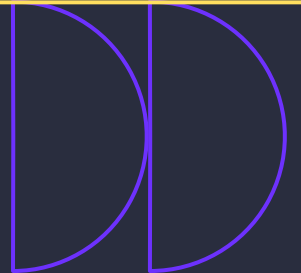
GPTQ

- Optimizes quantization using a calibrated dataset
- Overfits on calibrated dataset

Existing Methods

- Very Poor Accuracy
- High training cost
- Overfitting

Activation-aware Weight Quantization



Main Idea

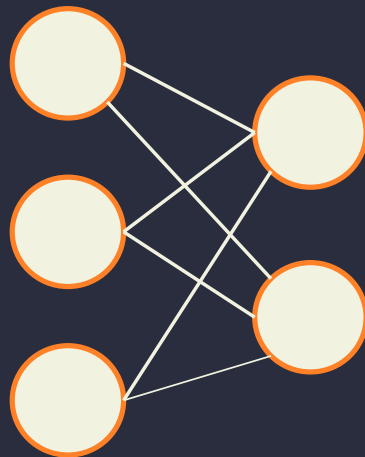
weights are not equally important

- Weights are not equally important for LLMs' performance
- Small fraction (0.1% - 1%) of "salient" weights → important weights
- Protecting these "important" weights, during quantization preserves accuracy

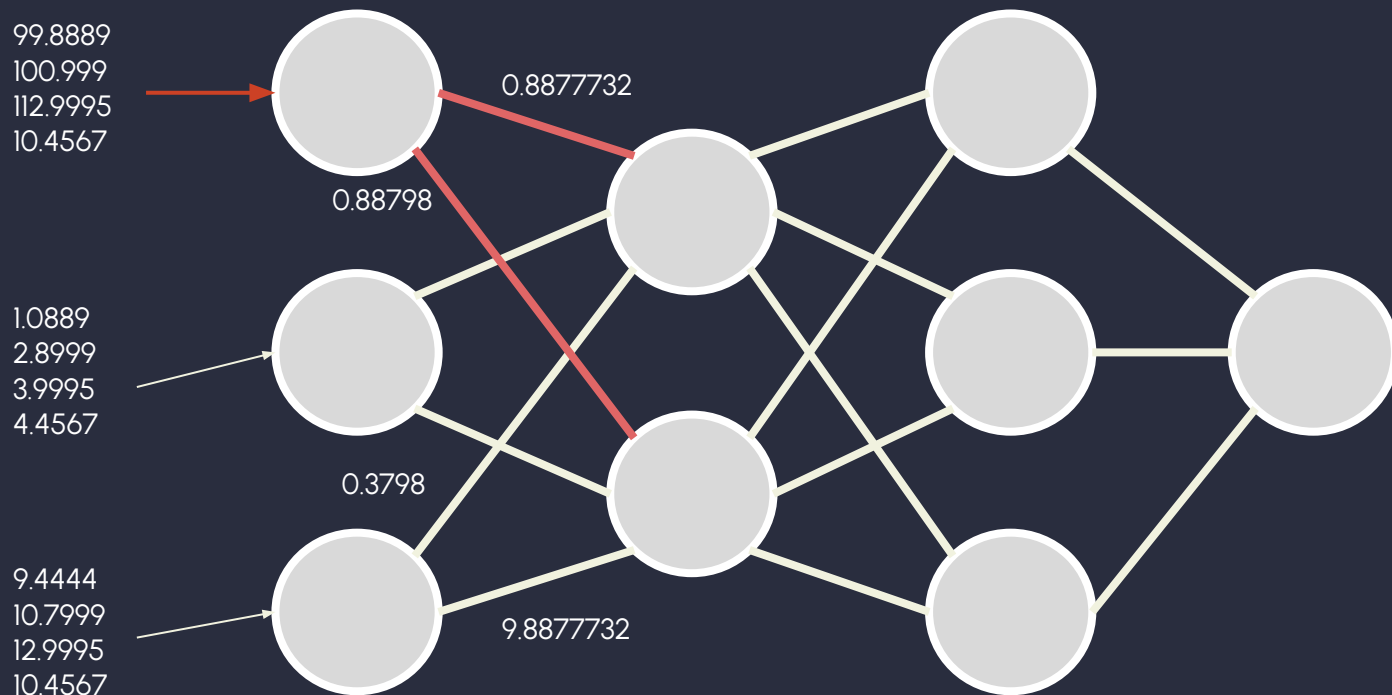
How to find salient weights?

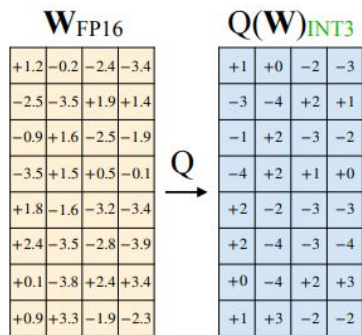
Answer: Activation-aware

- Use activation magnitude instead of weight size
- Channels with stronger or more varied activations are considered salient
- DO NOT QUANTIZE THESE SALIENT FEATURES

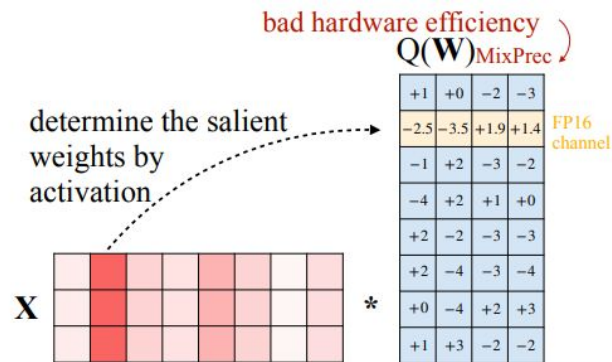


The highlighted weights are salient weights

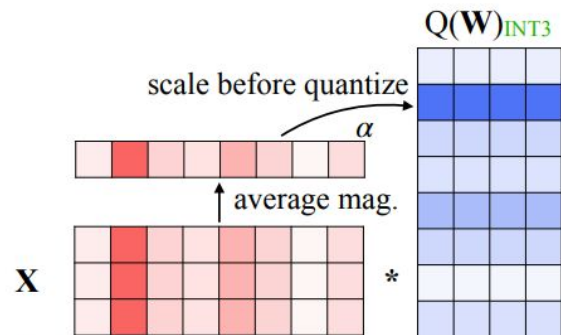




(a) RTN quantization (PPL 43.2)



(b) Keep 1% salient weights in FP16 (PPL 13.0)

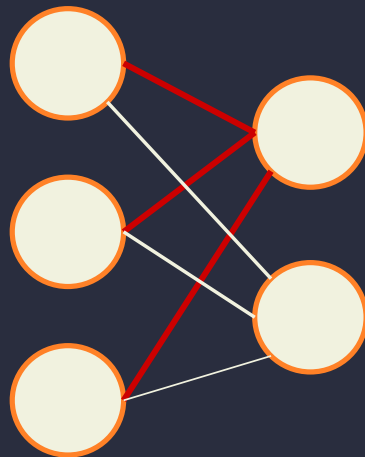


(c) Scale the weights before quantization (PPL 13.0)

Still Quantization loss?

Answer: Per-channel scaling

- Quantize non-salient weights to 4-bit precision
- Give an individual range to every channel
- Instead of using -1.0 to 1.0 for the whole neural net, make the range according to each channel's scale



Quantization & Dequantization

Consider this example:

4-bit quantization

Let's say range of weights is -7.0 to 7.0

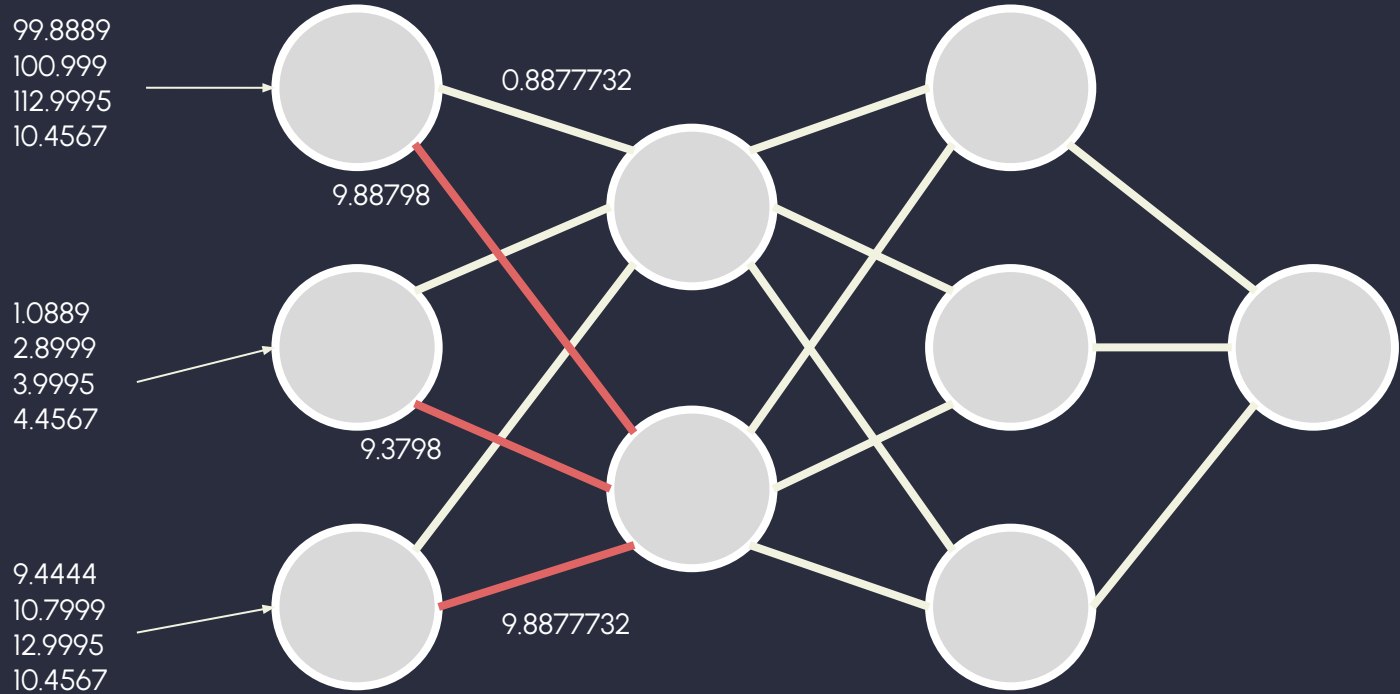
What is the Scale here? 1.0

Because every number is incremented by 1.0

But what is the flaw here? Global range

4-bit integer	Dequantized (approx)
0	- 7.0
1	- 6.0
2	- 5.0
...	...
14	6.0
15	7.0

Scale for the highlighted channel is 9 to 10



Mixed Precision



- Mixed-precision
- Difficult for GPUs to handle
- Kills performance

- Not mixed-precision
- Easy for GPUs to handle
- Improves performance

TinyChat

TinyChat

Purpose: Theoretical 4x smaller ≠ real speedup

Techniques Used:

- On-the-fly Dequantization
- Efficient 4-bit weight Packing
- Kernel Fusion

Result: Realizes ~3.2–3.3× speedup vs. FP16 models.

Dequantization

- Performed during the inference phase
- One of the steps in fused kernel
- Saves memory bandwidth
- Done by TinyChat
- One tile at a time

Weights matrix

[INT4	INT4	INT4]
[INT4	INT4	INT4]
[FP16	FP16	FP16]

Scale matrix

[scale_1]
[scale_2]
[] → no scale since not quantized

Weight packing

1	0	0	0	1	0	0	1	1	0	1	1	1	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

| - - - - - weight 1 - - - - - | - - - - - weight 2 - - - - - | - - - - - weight 3 - - - - - | - - - - - weight 4 - - - - - |

- Efficient memory usage
- Efficient usage of GPU resources
- Better performance

Kernel fusion

- Multiple GPU ops merged into one kernel
- Avoid overhead of switching kernels
- Reduces memory traffic → reduces latency
- Enables continuous compute without stalls

Example kernel function

```
load INT4  
apply scale  
convert to FP16  
matmul  
activation (maybe)  
store result
```

Experimental Results

Model (Accuracy↑)	VQAv2	GQA	VizWiz	SQA-I	VQA-T	POPE	MME	MMB	SEED	llava-bench	MM-Vet
VILA-7B	80.3	63.1	59.6	68.0	62.6	86.3	1489.4	69.8	61.7	75.2	35.1
VILA-7B-AWQ	80.1	63.0	57.8	68.0	61.9	85.3	1486.3	68.8	61.3	75.8	35.9
VILA-13B	80.5	63.6	63.1	70.5	64.0	86.3	1553.6	73.8	62.8	78.3	42.6
VILA-13B-AWQ	80.4	63.6	63.0	71.2	63.5	87.0	1552.9	73.6	62.2	77.6	42.0

OPT (PPL↓)	1.3B	2.7B	6.7B	13B	30B
FP16	14.62	12.47	10.86	10.13	9.56
RTN	119.47	298.00	23.54	46.04	18.80
1% FP16	16.91	13.69	11.39	10.43	9.85
$s = 2$	18.63	14.94	11.92	10.80	10.32
AWQ	16.32	13.58	11.39	10.56	9.77

Visual Results



W4-RTN: A model airplane **flying in the sky**.

W4-AWQ: Two toy airplanes **sit on a grass field**.



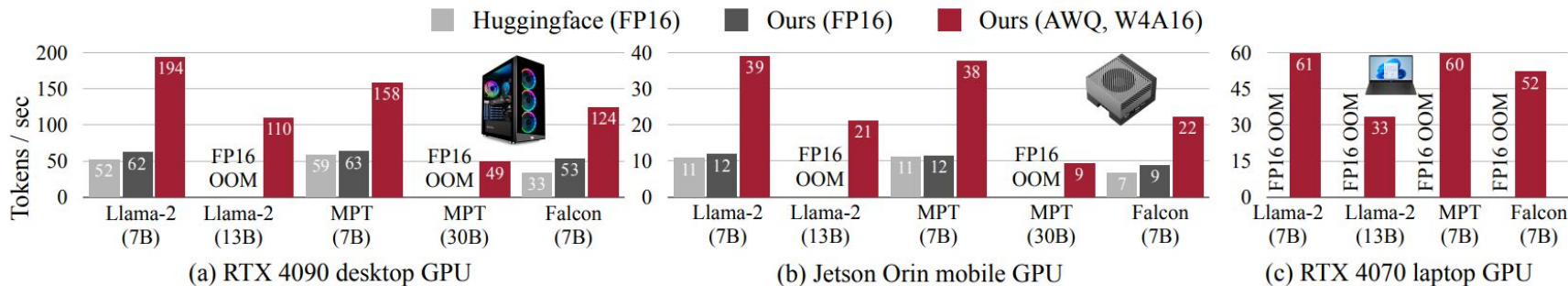
W4-RTN: A man is **holding a baby elephant** in his arms.

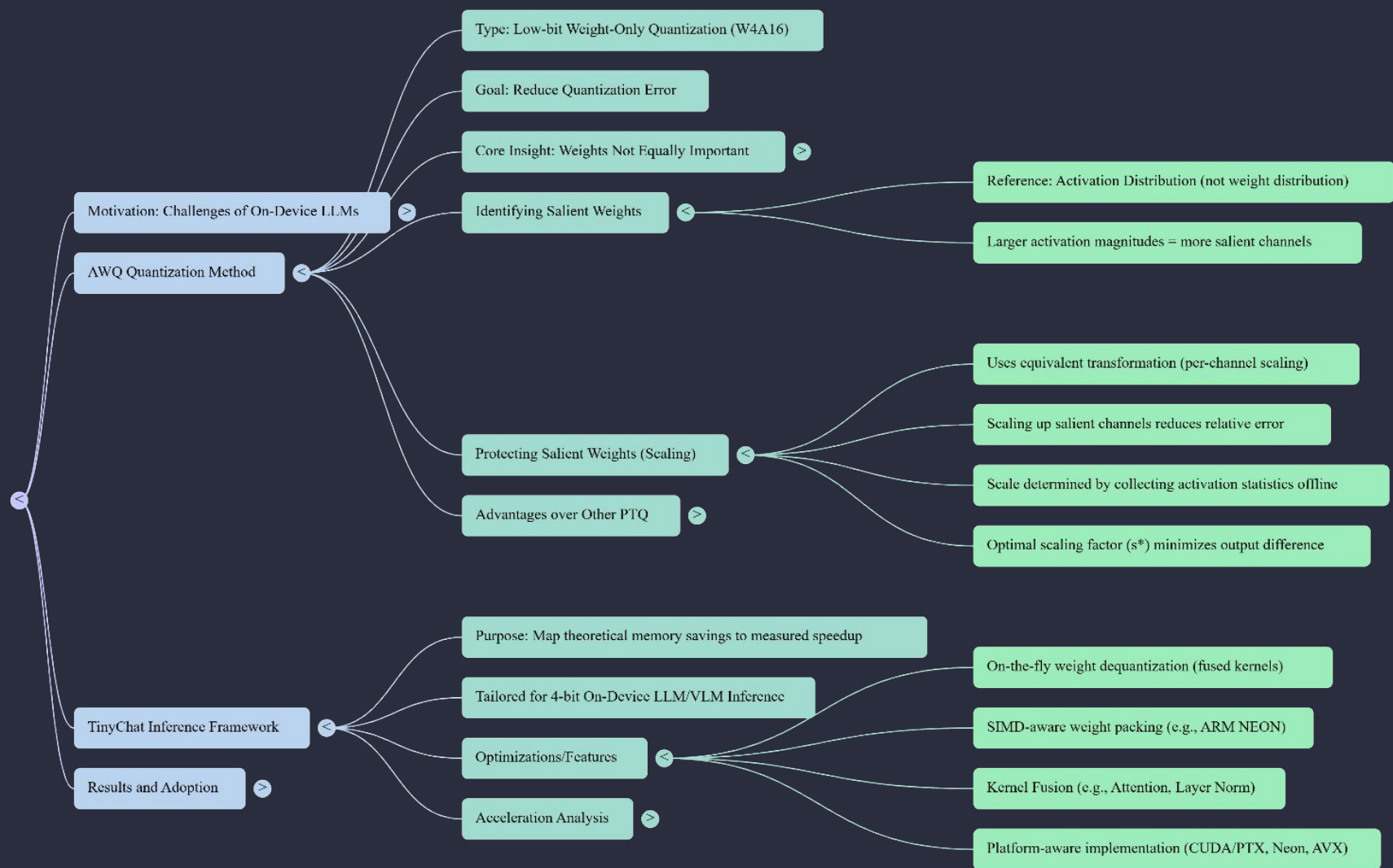
W4-AWQ: A man and his daughter **pose with an elephant**.



W4-RTN: **A man and a dog** walking past some bushes.

W4-AWQ: **Two dogs** are walking on the street.





Three teal semi-circles of increasing size are arranged horizontally on the left side of the slide. The first is the smallest, the second is medium, and the third is the largest, partially overlapping the text.

That's it.